



Comparativa: Exploración de modelos de inteligencia artificial en procesos de diseño de materiales.

Proyecto Integrador

Patrocinadores:

Dr. Juan Arturo Nolasco Flores

Dr. Samuel Lara Ávila

Integrantes:

Javier González Aces - A01274650

José Alejandro Cortés Pérez - A01795033

Osiris Xcaret Saavedra Solís - A01795992

Fecha de entrega: 15 de febrero del 2026

Baseline Alchemi

Los algoritmos que se puede utilizar como baseline: Dispersiones (interacciones), electrostática y vecindarios

Características para el modelo generado/rendimiento del modelo y aumentar la complejidad: Sistema de forma aleatoria por complejidad computacional

Métrica adecuada/ desempeño a obtener: Calculo de la distancia entre los vecindarios

Los 3 grande de Alchemi: Se evalúan los 3 principales componentes de Alchemi que consisten en las dispersiones (interacciones), electrostática y vecindarios (distancias, acomodados aleatorios (tiempos computacionales distintos) y verificación), se interpreta el código de Alchemi ToolKit Ops de la siguiente manera:

Importamos el módulo

```
from nvalchemiops.neighborlist.neighborlist import neighbor_list
```

Cargar los archivos

with open("./dimer.xyz") as f:

```
    lines = f.readlines()
```

```
    num_atoms = int(lines[0])
```

```
    coords = torch.zeros(num_atoms, 3, device=device, dtype=dtype)
```

```
    atomic_numbers = torch.zeros(num_atoms, device=device, dtype=torch.int32)
```

Lista de los vecindarios

```
neighbor_matrix, num_neighbors_per_atom, neighbor_matrix_shifts = neighbor_list(
```

```
    coords,
```

```
    cutoff=20.0, # Interaction cutoff in Angstrom
```

```
    cell=cell,
```

```
    pbc=pbc,
```

```
    method="cell_list", # O(N) cell list algorithm
```

```
    max_neighbors=64, # Maximum neighbors per atom
```

```
)
```

Calculada la matrix de los vecindarios

```
energies, forces, coord_num = d3_module(
```

```

    positions=coords,
    numbers=atomic_numbers,
    neighbor_matrix=neighbor_matrix,
)

```

Procesamiento de cristales

```
BATCH_SIZE = 16 # Number of structures to process simultaneously
```

```
CUTOFF = 25.0 # Interaction cutoff in Angstrom
```

Cálculo de coulomb electrostatic

Ingresa la separación de las cargas

```

positions = torch.tensor(
    [
        [0.0, 0.0, 0.0],
        [2.0, 0.0, 0.0],
    ],
    dtype=torch.float64,
    device=device,
)

```

Ingresan las dos cargas como tensores

```
charges = torch.tensor([1.0, -1.0])
```

Construcción de listas de vecindarios

Se establecen los tamaños de un sistema

```
small_num_atoms = 100
```

```
small_box_size = 10.0
```

Crea el Sistema de forma aleatoria

```
small_positions = (
```

```
    torch.rand(small_num_atoms, 3, dtype=dtype, device=device) * small_box_size
)
small_cell = (torch.eye(3, dtype=dtype, device=device) * small_box_size).unsqueeze(0)
```

Cálculo de la distancia entre los vecindarios

```
neighbor_list, neighbor_ptr, shifts = cell_list(
    neighbor_list, neighbor_ptr, shifts = cell_list(
        small_positions, cutoff, small_cell, pbc, return_neighbor_list=True
    )
```